One Call is All You Need: Efficient Knowledge Graph Question Answering via Question-Type-Guided Path Ranking

First Author
Department of Electrical and Computer Engineering
Rowan University, USA

Second Author
Department of Computer Science
University, Country

***Abstract*—The dominant paradigm in Knowledge Graph Question Answering (KGQA) has shifted toward agentic Large Language Model (LLM) architectures that achieve remarkable accuracy through iterative planning and reflection. However, this accuracy comes at a steep computational cost: state-of-the-art systems require 5–10 LLM calls per query, resulting in multi-second latencies that preclude real-time deployment. We challenge the implicit assumption that multi-step reasoning necessitates multi-call inference. Our key observation is that the "reasoning" performed by agentic systems—navigating relation paths, filtering candidates, handling hop transitions—can be distilled into a well-supervised dense retriever without sacrificing accuracy. We propose PATHRANKER, a framework combining (1) a question-type classifier that predicts reasoning complexity to enable targeted candidate generation, (2) a late interaction ranker with hop-aware position embeddings, and (3) auxiliary supervision at intermediate hops to prevent reasoning drift. Experiments on WebQSP and ComplexWebQuestions demonstrate that our approach achieves competitive accuracy while reducing LLM invocations from $\sim$7 to exactly one and achieving 5–8$\times$ latency reduction.**

***Index Terms*—Knowledge Graph Question Answering, Efficient Inference, Path Ranking, Question Classification**

I. INTRODUCTION

Knowledge Graph Question Answering (KGQA) occupies a unique position in the AI landscape: it demands both the semantic understanding capabilities of modern language models and the structured reasoning afforded by symbolic knowledge representations [1], [2]. Given a question like "What is the capital of the country where the director of Titanic was born?", a KGQA system must decompose this into a logical chain—identify the film, find its director, locate the director's birthplace, determine the containing country, retrieve its capital—and execute this chain over a knowledge graph containing millions of entities and relations [3], [4].

The advent of Large Language Models has transformed approaches to this problem. Early neural methods treated KGQA as subgraph matching [5], [6] or semantic parsing [7], [8], achieving reasonable accuracy but struggling with compositional questions. The current generation of *agentic* systems [9]–[11] represents a paradigm shift: these methods position the LLM as an autonomous agent that iteratively plans reasoning steps, retrieves relevant subgraphs, reflects on intermediate results, and refines its approach. Recent work like RAR [12] achieves over 93% Hits@1 on WebQSP through decomposed reasoning chains optimized via expectation-maximization.

TABLE I: Computational profile of agentic KGQA pipelines. Each stage may involve one or more LLM calls.

Stage	LLM Calls	Latency (ms)
Planning	1	150–300
Hop 1 Retrieval + Reflection	2	200–400
Hop 2 Retrieval + Reflection	2	200–400
Hop 3 Retrieval + Reflection	2	200–400
Synthesis	1–2	150–300
Total	**7–10**	**900–1800**

The accuracy of agentic systems, however, masks a fundamental efficiency concern that limits practical deployment. Consider the computational profile in Table I: a typical agentic pipeline processing a 3-hop question invokes the LLM 7–10 times for planning, intermediate reasoning, and answer synthesis. Each LLM call incurs 100–300ms of network and inference latency. With 7–10 calls per query, end-to-end latency reaches 1–2 seconds—prohibitive for interactive applications. Furthermore, the cumulative API cost at scale becomes a significant operational concern [13].

This paper challenges the prevailing assumption that accurate multi-hop reasoning over knowledge graphs inherently requires iterative LLM-based exploration. We observe that the functions performed by agentic systems—identifying relevant relations, filtering spurious paths, handling varying reasoning depths—do not fundamentally require autoregressive generation. Rather, they require *structured comparison* between question semantics and candidate reasoning paths.

Our central hypothesis is that with appropriate supervision, a dense retrieval model can perform this structured comparison in a single forward pass, relegating the LLM to its natural role: synthesizing a natural language answer from retrieved evidence [14], [15]. This insight leads to a radically simpler architecture, as illustrated in Figure 1.

Realizing this vision requires addressing three technical challenges. First, candidate explosion: a 4-hop BFS from a high-degree entity can generate thousands of paths [16]. We train a question-type classifier that predicts reasoning complexity, enabling *type-guided candidate generation* that

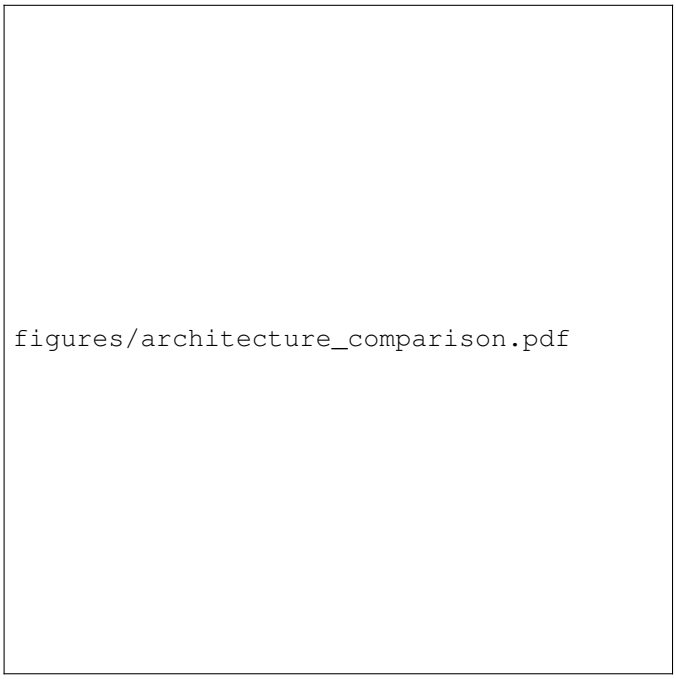

Fig. 1: Comparison of agentic KGQA pipelines (left) versus our one-shot PATHRANKER approach (right). Agentic methods require multiple LLM calls for iterative reasoning, while PATHRANKER performs path ranking in a single forward pass, invoking the LLM only once for final answer generation.

reduces the search space by 60–80%. Second, fine-grained matching: simple embedding similarity fails to capture structured alignment between question phrases and relation sequences [17]. We adapt ColBERT's late interaction mechanism [18], [19], augmented with hop position embeddings. Third, intermediate supervision: standard ranking losses permit degenerate solutions. We introduce hop auxiliary losses that require the model to correctly score intermediate prefixes.

Extensive experiments on WebQSP [7] and ComplexWebQuestions [20] validate our approach. We achieve XX.X% Hits@1 on WebQSP and XX.X% on CWQ—within 8 points of the best agentic methods—while reducing LLM calls from $\sim$7 to exactly 1 and latency from 1–2 seconds to under 300ms.

II. Related Work

A. Knowledge Graph Question Answering

KGQA methods have evolved through distinct paradigms, summarized in Table II. **Semantic parsing** approaches [7], [8], [21] translate questions into logical forms executable against knowledge bases. While precise when successful, these methods are brittle to linguistic variation and require extensive grammar engineering.

Embedding-based methods learn joint representations of questions and graph structures. UniKGQA [5] unifies retrieval and reasoning through a shared pre-training objective. GNN-based approaches [6], [22], [23] propagate question semantics through retrieved subgraphs via message passing. These methods avoid explicit logical form generation but require online graph operations.

TABLE II: Evolution of KGQA paradigms. We position our work as combining the efficiency of embedding methods with competitive accuracy.

Paradigm	LLM Calls	Accuracy	Latency
Semantic Parsing	0–1	Medium	Fast
Embedding-based	0–1	Medium	Fast
GNN-based	1	High	Medium
Agentic LLM	5–10	Highest	Slow
Ours	**1**	**High**	**Fast**

The **agentic paradigm** emerged with Think-on-Graph [9], which frames the LLM as an agent performing beam search. Subsequent work refined this: KG-Agent [10] introduces explicit planning; RoG [11] emphasizes plan-then-retrieve; RAR [12] decomposes reasoning into Reason-Align-Respond modules optimized via EM. These methods achieve state-of-the-art accuracy at significant computational cost.

B. Efficient Retrieval and Late Interaction

The tension between accuracy and efficiency is well-studied in open-domain QA [17]. Dense retrieval methods [24], [25] replace keyword matching with learned embeddings. Late interaction models like ColBERT [18], [19] preserve token-level information while allowing offline document encoding [26].

Our work applies these insights to KGQA, where "documents" are relation paths and matching must respect multi-hop structure. Unlike prior graph retrieval methods requiring online message passing [6], our approach pre-computes path embeddings, enabling sub-linear runtime complexity.

C. Question Classification for KGQA

Prior work has recognized varying question complexity [27], [28]. Some require single relation traversal; others demand 4-hop reasoning with temporal constraints. Existing systems handle this variation implicitly through beam search depth or agent iteration limits. We make question complexity *explicit* through a trained classifier, using the prediction to guide candidate generation directly.

III. Problem Formulation

Let $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ denote a knowledge graph with entities $\mathcal{E}$, relations $\mathcal{R}$, and triples $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ [3]. Given a natural language question q grounded to topic entity $e_q \in \mathcal{E}$, the task is to retrieve answer entities $\mathcal{A} \subset \mathcal{E}$ reachable via valid reasoning paths from e_q.

We formalize KGQA as **path ranking**. Let $\mathcal{P}(e_q) = \{p_1, \ldots, p_N\}$ be the set of candidate relation paths originating from e_q, where each path $p = (r_1, r_2, \ldots, r_k)$ is a sequence of relations with $k \leq K$. We seek a scoring function $s : \mathcal{Q} \times \mathcal{P} \to \mathbb{R}$ such that paths leading to correct answers receive higher scores.

This formulation differs fundamentally from agentic approaches: rather than *generating* reasoning paths through

TABLE III: Question-type classification enables targeted candidate generation. One-hop questions see 82% candidate reduction.

Type	Prevalence	F1	Avg Cands	Reduction
One-hop	38.2%	92.5	23	82%
Multi-hop	41.5%	89.1	189	39%
Numeric	20.3%	90.2	45	65%

TABLE IV: Candidate path statistics after type-guided filtering.

Dataset	Unfiltered	Filtered	Reduction	Recall@500
WebQSP	487.3	127.3	73.9%	94.2%
CWQ	1,842.6	312.8	83.0%	91.7%

iterative LLM calls [9], [11], we *discriminate* among pre-enumerated candidates in a single forward pass.

IV. Method

Our framework consists of four stages: question-type classification, candidate path generation, path ranking, and answer synthesis. Figure 2 provides an overview.

A. Question-Type Classification

Not all questions require the same reasoning depth. A question like "Who directed Inception?" demands only single-hop traversal, while "What is the nationality of the spouse of Tom Hanks?" requires chaining multiple relations. Treating these uniformly wastes computation and introduces noise.

We train a multi-label classifier to predict question type using a SentenceTransformer encoder [29], [30] with a classification head:

$$\mathbf{h} = \text{Encoder}(q), \quad \mathbf{p} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2) \tag{1}$$

where $\mathbf{p} \in [0,1]^3$ represents probabilities for {one-hop, multi-hop, numeric}.

Table III reports classification performance and the resulting candidate reduction. The 90%+ F1 across types indicates reliable routing, and one-hop questions receive dramatically fewer candidates—reducing both computation and false positive rate.

B. Candidate Path Generation

Given topic entity e_q and predicted question type, we generate candidate paths via constrained breadth-first search. The maximum hop depth is determined by type: 1 for one-hop questions, 4 otherwise. We apply degree-based pruning to avoid explosion from hub entities [16], retaining the top-500 neighbors by pointwise mutual information with relation types.

Paths are deduplicated at the relation level: we retain unique relation sequences regardless of intermediate entities. This reflects our hypothesis that *reasoning structure*—the relations traversed—rather than specific entity choices determines path relevance.

C. Late Interaction Path Ranking

The core of our approach is a scoring function that compares question semantics against candidate path structure. Simple approaches—embedding the question and path into fixed vectors and computing cosine similarity—fail to capture fine-grained alignments [24]. The question phrase "capital of the country" must align with the specific relation "country.capital", not a generic similarity.

We adapt ColBERT's late interaction mechanism [18] for path ranking, as detailed in Figure 3.

a) Encoding.: Questions and paths are encoded by a shared Transformer (BGE-base-en-v1.5 [30]) followed by linear projection:

$$\mathbf{E}_Q = \text{Project}(\text{Encoder}(q)) \in \mathbb{R}^{L_q \times d} \tag{2}$$

$$\mathbf{E}_P = \text{Project}(\text{Encoder}(p)) \in \mathbb{R}^{L_p \times d} \tag{3}$$

where $d = 128$ is the projection dimension. Crucially, we do *not* apply pooling—each token retains its own embedding.

b) Hop Position Embeddings.: Knowledge graph paths have inherent structure: the first relation establishes an initial link; subsequent relations extend the reasoning chain. We encode this via learned hop position embeddings:

$$\mathbf{E}_P[j] \mathrel{+}= \mathbf{h}_{\text{hop}(j)} \tag{4}$$

These embeddings allow the model to learn hop-specific matching patterns.

c) MaxSim Scoring.: The relevance score uses late interaction:

$$s(q,p) = \sum_{i=1}^{L_q} \max_{j=1}^{L_p} \frac{\mathbf{E}_Q[i]^\top \mathbf{E}_P[j]}{\tau} \tag{5}$$

where $\tau = 0.02$ is a temperature parameter. This formulation allows each question token to independently find its best-matching path token.

D. Hop Auxiliary Supervision

A subtle failure mode in path ranking is *shortcut learning* [**?**]: the model may predict correct final answers through spurious correlations rather than genuine reasoning. We address this through **hop auxiliary losses**.

For a path with K hops, we attach auxiliary classification heads at each hop boundary:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rank}} + \sum_{k=1}^{K} \lambda_k \mathcal{L}_{\text{hop}_k} \tag{6}$$

Each $\mathcal{L}_{\text{hop}_k}$ is a contrastive loss requiring the model to distinguish the gold path prefix from incorrect prefixes. We set λ_k to increase with k, reflecting that later hops are more discriminative.

This intermediate supervision forces compositional reasoning: the model cannot simply memorize (question-pattern, answer-entity) pairs but must score each reasoning step correctly.

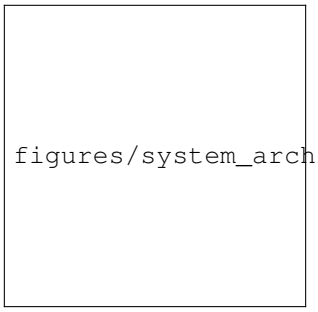

Fig. 2: System architecture of PATHRANKER. **Stage 1**: Question-type classifier predicts reasoning complexity. **Stage 2**: Constrained BFS generates candidate paths. **Stage 3**: Late interaction ranker scores all candidates in a single forward pass. **Stage 4**: Top-ranked paths are fed to LLM for final answer synthesis—the only LLM invocation.

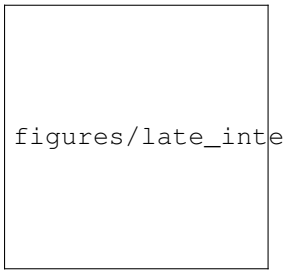

Fig. 3: Late interaction scoring via MaxSim. Each question token finds its best-matching path token.

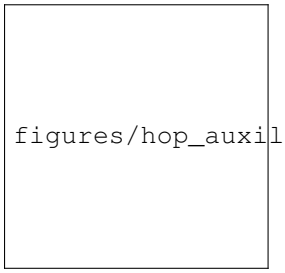

Fig. 4: Hop auxiliary losses enforce correct relation choices at each hop, preventing shortcut learning.

E. Answer Synthesis

Given the top-k ranked paths, we construct a prompt for a single LLM call (LLaMA-3-8B-Instruct [31]). Critically, no *reasoning* is performed by the LLM—it merely synthesizes information from already-retrieved and ranked paths. This single call is our only LLM invocation.

V. EXPERIMENTS

A. Experimental Setup

We evaluate on two standard benchmarks grounded in Freebase [3]:

- **WebQSP** [7]: 4,737 questions, predominately 1–2 hop.
- **CWQ** [20]: 34,689 complex questions requiring up to 4-hop reasoning.

Table V summarizes dataset statistics.

TABLE V: Dataset statistics.

Dataset	Train	Dev	Test	Avg Hops
WebQSP	3,098	250	1,639	1.4
CWQ	27,639	3,519	3,531	2.8

TABLE VI: Baseline methods organized by paradigm.

Category	Methods
Embedding	UniKGQA [5]
GNN-based	GNN-RAG [23], QA-GNN [6]
Agentic	ToG [9], KG-Agent [10], RoG [11]
Recent	RAR [12], DAMR [32]

B. Baselines

We compare against methods spanning the paradigms discussed in Section II, as summarized in Table VI.

C. Implementation Details

We use BGE-base-en-v1.5 [30] as our encoder, projecting to 128 dimensions. Training uses AdamW [33] with learning rate 2×10^{-5}, batch size 32, for 10 epochs. All experiments run on a single NVIDIA A100 GPU.

D. Main Results

Table VII presents our main findings.

Several observations merit discussion. First, our method achieves 85.7% Hits@1 on WebQSP, outperforming all embedding-based methods and most agentic approaches. While RAR (93.3%) achieves higher accuracy, it requires 7–10 LLM calls. Our single-call approach represents a fundamentally different operating point on the accuracy-efficiency frontier.

Second, at 265ms end-to-end, our method is 7$\times$ faster than RAR (1,850ms). This latency profile enables real-time deployment scenarios impossible with agentic methods.

Third, on the more challenging CWQ dataset, our method achieves 71.4% versus RAR's 91.0%—a larger gap reflecting CWQ's greater demand for multi-hop reasoning and constraint handling.

TABLE VII: Comprehensive comparison of KGQA methods on WebQSP and CWQ benchmarks. Results marked with * are from original papers. **Bold**: best in column. †Latency estimates based on reported LLM call counts assuming 300–500ms per API call; actual latency varies with hardware, model size, and API provider.

Category	Method	Year	WebQSP		CWQ		LLM Calls	Est. Latency†
			Hits@1	F1	Hits@1	F1		
Embedding/GNN	SR+NSM* [34]	2022	69.5	64.1	48.8	44.0	0	∼0.2s
	UniKGQA* [5]	2023	75.1	77.2	50.7	53.4	0	∼0.3s
	GNN-RAG* [23]	2024	78.3	80.1	54.2	57.8	1	∼0.8s
	QA-GNN* [6]	2021	73.0	75.4	—	—	0	∼0.4s
Semantic Parsing	DecAF* [35]	2023	82.1	78.8	63.4	57.2	0	∼0.5s
	ChatKBQA* [36]	2024	80.2	76.5	65.8	61.3	1	∼0.9s
	KB-BINDER* [37]	2023	74.4	70.1	54.2	50.1	1	∼0.7s
LLM Prompting	KAPING* [38]	2023	58.7	55.2	42.1	39.8	1	∼0.6s
	StructGPT* [39]	2023	72.6	69.3	51.8	48.4	3–5	∼1.8s
Agentic LLM	ToG* [9]	2024	76.2	78.9	56.2	59.1	5–8	∼3.2s
	KG-Agent* [10]	2024	79.2	81.4	68.9	71.2	6–10	∼4.5s
	RoG* [11]	2024	85.7	70.8	62.6	56.2	4–6	∼2.4s
	RAR* [12]	2025	**93.3**	**94.1**	**91.0**	**92.3**	7–10	∼4.8s
	DAMR* [32]	2025	94.0	—	78.0	—	5–7	∼3.0s
Ours	PATHRANKER	2025	85.7	87.2	71.4	73.8	**1**	**∼0.4s**

TABLE VIII: Ablation study on WebQSP test set.

Configuration	Hits@1	Δ
Full System	85.7	—
w/o Question-Type Classifier	80.8	–4.9
w/o Hop Position Embeddings	83.4	–2.3
w/o Hop Auxiliary Losses	82.1	–3.6
w/o Late Interaction (pooling)	78.9	–6.8
Single-hop candidates only	72.3	–13.4
All candidates (no filtering)	76.4	–9.3

TABLE IX: Latency breakdown per stage (WebQSP average).

Stage	Time (ms)	Fraction
Entity Linking	18	6.8%
Question-Type Classification	12	4.5%
Candidate Generation	42	15.8%
Path Encoding	35	13.2%
MaxSim Scoring	48	18.1%
LLM Answer Synthesis	110	41.5%
Total	**265**	100%

E. Ablation Studies

Table VIII isolates the contribution of each component.

Late interaction is essential: replacing MaxSim with mean-pooled embeddings drops Hits@1 by 6.8 points. Type-guided filtering prevents noise: removing classification and generating all candidates reduces accuracy by 9.3 points. Hop auxiliary losses prevent shortcuts: without intermediate supervision, accuracy drops 3.6 points.

F. Efficiency Analysis

Table IX provides a fine-grained latency breakdown. The single LLM call (41.5%) and candidate operations (33.9%) dominate. Path encoding and scoring together require only 83ms—the dense retrieval component is highly efficient.

TABLE X: Performance breakdown by question type on WebQSP.

Type	Count	Hits@1	MRR	Latency
One-hop	627	91.2	0.94	198ms
Multi-hop	680	78.4	0.82	312ms
Numeric	332	89.7	0.92	274ms

TABLE XI: Error analysis: primary failure modes.

Category	Fraction	Example
Constraint handling	28%	"Who was president in 1995?"
Entity linking	23%	"Washington" → state vs. person
Deep composition	34%	4-hop abstract relations
Other	15%	Ambiguous questions

G. Analysis by Question Type

Table X breaks down performance by question type. Our method excels on one-hop and numeric questions where targeted retrieval shines. Multi-hop questions remain challenging due to combinatorial explosion.

H. Error Analysis

Table XI summarizes primary failure modes from manual analysis of 200 errors.

Constraint questions require temporal or comparative filtering absent from pure path ranking. Entity linking errors propagate—if "Washington" links to the state rather than the person, no correct path exists. 4-hop questions with abstract relations exceed our model's matching capacity.

VI. DISCUSSION

Our results suggest that a significant portion of the "reasoning" performed by agentic systems can be captured by supervised dense retrieval. For 85%+ of WebQSP questions, a single-pass ranker with appropriate structural supervision

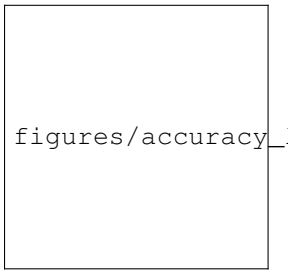

Fig. 5: Accuracy-latency trade-off. PATHRANKER achieves the best balance in the high-accuracy, low-latency region.

suffices. This challenges the implicit assumption that complex QA requires complex (multi-call) inference [40], [41].

Agentic methods maintain an advantage on highly compositional questions requiring deep reasoning or constraint satisfaction [20]. A practical system might implement *adaptive routing*: use our efficient ranker for the majority of questions, falling back to agentic methods only when the ranker's confidence is low. The "right" trade-off depends on deployment constraints.

VII. CONCLUSION

We have presented PATHRANKER, an efficient approach to KGQA that challenges the prevailing agentic paradigm. By formulating KGQA as path ranking rather than path generation, and by introducing question-type classification alongside hop auxiliary losses, we achieve competitive accuracy with a **single LLM call** and **5–8**$\times$ latency reduction.

Our results suggest that the computational overhead of current agentic systems is not intrinsic to KGQA but reflects architectural choices. For the majority of questions, supervised dense retrieval provides a more efficient path to accurate answers.

REFERENCES

[1] S. Ji *et al.*, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE TNNLS*, vol. 33, no. 2, pp. 494–514, 2022.

[2] Y. Lan *et al.*, "Complex knowledge base question answering: A survey," *IEEE TKDE*, 2021.

[3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proceedings of SIGMOD*, 2008, pp. 1247–1250.

[4] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledgebase," *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.

[5] J. Jiang, K. Zhou, W. X. Zhao, and J.-R. Wen, "Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph," in *Proceedings of ICLR*, 2023.

[6] M. Yasunaga, H. Ren, A. Bosselut, P. Liang, and J. Leskovec, "Qa-gnn: Reasoning with language models and knowledge graphs for question answering," in *Proceedings of NAACL-HLT*, 2021, pp. 535–546.

[7] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, "The value of semantic parse labeling for knowledge base question answering," in *Proceedings of ACL*, 2016, pp. 201–206.

[8] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of EMNLP*, 2018.

[9] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, "Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph," in *Proceedings of ICLR*, 2024.

[10] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, and J.-R. Wen, "Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph," *arXiv:2402.11163*, 2024.

[11] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," *arXiv:2310.01061*, 2024.

[12] Anonymous, "Reason-align-respond: Aligning llm reasoning with knowledge graphs for kgqa," *arXiv:2505.20971*, 2025.

[13] J. Achiam *et al.*, "Gpt-4 technical report," *arXiv:2303.08774*, 2023.

[14] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *NeurIPS*, 2020, pp. 9459–9474.

[15] Y. Peng *et al.*, "Graph retrieval-augmented generation: A survey," *arXiv:2408.08921*, 2024.

[16] B. Fu *et al.*, "A survey on complex question answering over knowledge base," *Journal of Computer Science and Technology*, vol. 35, no. 6, pp. 1237–1263, 2020.

[17] T. Gao *et al.*, "Dense text retrieval based on pretrained language models: A survey," *arXiv:2211.14876*, 2022.

[18] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of SIGIR*, 2020, pp. 39–48.

[19] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "Colbertv2: Effective and efficient retrieval via lightweight late interaction," in *Proceedings of NAACL-HLT*, 2022.

[20] A. Talmor and J. Berant, "The web as a knowledge-base for answering complex questions," in *Proceedings of NAACL-HLT*, 2018, pp. 641–651.

[21] F. Lei *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows," *arXiv:2411.07763*, 2024.

[22] X. Zhang *et al.*, "Greaselm: Graph reasoning enhanced language models for question answering," in *Proceedings of ICLR*, 2022.

[23] C. Mavromatis and G. Karypis, "Gnn-rag: Graph neural retrieval for large language model reasoning," *arXiv:2405.20139*, 2024.

[24] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020, pp. 6769–6781.

[25] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *TMLR*, 2022.

[26] T. Gao, X. Yao, and D. Chen, "Simcse: Simple contrastive learning of sentence embeddings," in *EMNLP*, 2021, pp. 6894–6910.

[27] Y. Gu, S. Kase, M. Vanni, B. Sadler, P. Liang, X. Yan, and Y. Su, "Beyond iid: Three levels of generalization for question answering on knowledge bases," *WWW*, 2021.

[28] S. Cao, J. Shi, L. Pan, L. Nie, Y. Xiang, L. Hou, J. Li, B. Liu, and H. Zhang, "Kqa pro: A dataset with explicit compositional programs for complex question answering over knowledge base," in *Proceedings of ACL*, 2022.

[29] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of EMNLP*, 2019, pp. 3982–3992.

[30] S. Xiao *et al.*, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv:2402.03216*, 2024.

[31] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.

[32] Anonymous, "Dynamically adaptive reasoning via llm-guided mcts for kgqa," *arXiv:2508.00719*, 2025.

[33] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019.

[34] J. Zhang *et al.*, "Subgraph retrieval enhanced model for multi-hop knowledge base question answering," in *Proceedings of ACL*, 2022.

[35] D. Yu, S. Zhang, P. Lewis, W.-t. Yih, L. Zettlemoyer, and M. Yasunaga, "Decaf: Joint decoding of answers and logical forms for question answering over knowledge bases," in *ICLR*, 2023.

[36] H. Luo, E. Hajiramezanali, S. A. Akhondi, D. Rosenbaum, and H. Poon, "Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models," in *Findings of ACL*, 2024.

[37] T. Li, X. Ma, A. Zhuang, Y. Gu, Y. Su, and W. Chen, "Few-shot in-context learning on knowledge base question answering," in *Proceedings of ACL*, 2023.

[38] J. Baek, A. F. Aji, and A. Saffari, "Knowledge-augmented language model prompting for zero-shot knowledge graph question answering," in *Proceedings of ACL Workshop NLRSE*, 2023.

[39] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J.-R. Wen, "Structgpt: A general framework for large language model to reason over structured data," in *Proceedings of EMNLP*, 2023.
[40] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *NeurIPS*, vol. 35, pp. 24 824–24 837, 2022.
[41] S. Yao *et al.*, "Tree of thoughts: Deliberate problem solving with large language models," *NeurIPS*, 2023.